

Contents

1	Clawdbot Plugin Development Guide	2
1.1	Table of Contents	2
1.2	Understanding the Agent Runtime Environment	2
1.2.1	What Is the Agent Runtime?	2
1.2.2	Architecture Diagram	2
1.2.3	Why Is It Isolated?	3
1.3	Why Binaries Aren't Available	3
1.3.1	The Problem	3
1.3.2	Why This Fails	3
1.3.3	Real-World Example	4
1.4	Installing Inside the Runtime	4
1.4.1	What Does "Installing Inside the Runtime" Mean?	4
1.4.2	Approach 1: Docker Container Runtime	4
1.4.3	Approach 2: Virtual Environment	5
1.4.4	Approach 3: Whitelist Allowed Binaries	5
1.4.5	Why These Approaches Are Complex	5
1.5	The Plugin Approach	6
1.5.1	Why Plugins Are Better	6
1.5.2	Architecture Comparison	6
1.5.3	How It Works	7
1.5.4	Benefits of Plugin Approach	8
1.6	Creating a Plugin	8
1.6.1	Plugin Structure	8
1.6.2	Minimal Plugin Example	9
1.6.3	Plugin Installation	11
1.7	Plugin Development Workflow	11
1.7.1	Development Setup: Using Symlinks	11
1.8	Example: Markdown to PDF Plugin	15
1.8.1	Use Case	15
1.8.2	Plugin Structure	15
1.8.3	Key Files	15
1.8.4	How It Works in Practice	17
1.9	Best Practices	18
1.9.1	When to Create a Plugin	18
1.9.2	Security Considerations	18
1.9.3	Error Handling	18
1.10	Summary	19
1.10.1	Key Takeaways	19
1.10.2	Architecture Benefits	20
1.10.3	When to Use Each Approach	20
1.11	Resources	20
1.12	Questions?	20

1 Clawdbot Plugin Development Guide

Understanding agent runtime environments, sandboxing, and why plugins are the right approach for system integration.

1.1 Table of Contents

1. Understanding the Agent Runtime Environment
 2. Why Binaries Aren't Available
 3. Installing Inside the Runtime
 4. The Plugin Approach
 5. Creating a Plugin
 6. Example: Markdown to PDF Plugin
-

1.2 Understanding the Agent Runtime Environment

1.2.1 What Is the Agent Runtime?

The **agent's runtime environment** is an isolated execution context where the agent runs its tools and commands. Think of it as a sandbox that:

- Executes agent commands (`bash`, `read`, `write`, etc.)
- Has limited access to the host system
- Enforces workspace boundaries
- Provides security isolation

1.2.2 Architecture Diagram

Host System (Your Mac/Server)

Full system access
All binaries available (`pandoc`, `git`, etc.)
No restrictions

Agent Runtime Environment (Sandbox)

Isolated execution context
Limited binaries (`node`, `basic shell`)
Workspace restrictions

When agent executes:

```
bash("pandoc file.md -o file.pdf")  
→ Error: pandoc: command not found
```

When agent executes:

```
bash("cat /Users/john/.ssh/id_rsa")
```

→ Error: outside workspace

1.2.3 Why Is It Isolated?

Security Reasons:

1. **Prevent unauthorized file access**
 - Agent can't read `.ssh` keys
 - Agent can't access `.aws` credentials
 - Agent can't modify system files
 2. **Limit damage from malicious commands**
 - Can't run `rm -rf /`
 - Can't install malware
 - Can't modify critical configs
 3. **Control resource usage**
 - Prevent CPU/memory abuse
 - Limit network access
 - Control disk I/O
 4. **Enforce workspace boundaries**
 - Agent only accesses designated directories
 - Can't traverse outside workspace
 - Protects user's other projects
-

1.3 Why Binaries Aren't Available

1.3.1 The Problem

When a user asks: "Convert this markdown to PDF"

What the agent tries:

```
// Agent's attempt
bash("pandoc input.md -o output.pdf")
```

What happens:

Error: pandoc: command not found

1.3.2 Why This Fails

1. **Runtime doesn't include pandoc**
 - Agent sandbox has minimal binaries
 - Only includes: `node`, `bash`, basic shell utils
 - External tools like `pandoc`, `pdflatex` not included
2. **PATH limitations**
 - Agent's PATH doesn't include `/opt/homebrew/bin`
 - Agent's PATH doesn't include `/Library/TeX/texbin`
 - Even if you add them to gateway startup, the agent subprocess doesn't inherit them

3. Sandbox isolation

- Agent runtime may use a separate process tree
- Environment variables don't propagate
- Binary paths not visible to sandboxed processes

1.3.3 Real-World Example

```
// Gateway starts with this PATH
export PATH="/opt/homebrew/bin:/Library/TeX/texbin:$PATH"

// Gateway spawns agent runtime
const agent = spawn('node', ['agent.js'], {
  env: { /* minimal env, PATH reset */ }
})

// Agent tries to run pandoc
agent.exec('pandoc file.md -o file.pdf')
// → pandoc: command not found
```

1.4 Installing Inside the Runtime

1.4.1 What Does “Installing Inside the Runtime” Mean?

Making binaries available **within the agent’s isolated execution environment**.

1.4.2 Approach 1: Docker Container Runtime

If the agent runs in Docker:

```
# Dockerfile for agent runtime
FROM node:22-alpine

# Install system binaries INSIDE the container
RUN apk add --no-cache \
    pandoc \
    texlive \
    texlive-xetex \
    git \
    python3

# Now agent's commands can access these binaries
WORKDIR /workspace
CMD ["node", "agent.js"]
```

Result: - Agent can run pandoc - Agent can run pdf_lat_ex - Container image is now 500MB+ (was 100MB) - Build time increased significantly - Platform-specific builds needed

1.4.3 Approach 2: Virtual Environment

If the agent uses isolated environments:

```
# Create isolated environment for agent
mkdir -p ~/.clawdbot/agent-runtime/bin

# Copy binaries into agent's environment
cp /opt/homebrew/bin/pandoc ~/.clawdbot/agent-runtime/bin/
cp /Library/TeX/texbin/* ~/.clawdbot/agent-runtime/bin/
cp /usr/bin/git ~/.clawdbot/agent-runtime/bin/

# Agent's PATH points to this directory
export PATH="$HOME/.clawdbot/agent-runtime/bin:$PATH"
```

Result: - Agent can run binaries - Binary dependencies missing (shared libraries) - Version conflicts with system - Hard to maintain and update

1.4.4 Approach 3: Whitelist Allowed Binaries

Modify clawdbot to allow specific system binaries:

```
// In clawdbot source code
const allowedBinaries = [
  '/usr/bin/node',
  '/usr/bin/python3',
  '/usr/bin/git',
  '/opt/homebrew/bin/pandoc',    // Add this
  '/Library/TeX/texbin/pdflatex', // Add this
  '/Library/TeX/texbin/xelatex', // Add this
];

function executeSandboxed(command: string) {
  const binary = command.split(' ')[0];
  const fullPath = which(binary);

  if (!allowedBinaries.includes(fullPath)) {
    throw new Error(`Binary not allowed: ${binary}`);
  }

  return exec(command);
}
```

Result: - Agent can run specific binaries - Requires modifying clawdbot source - Security implications (larger attack surface) - Platform-specific paths

1.4.5 Why These Approaches Are Complex

Problems:

1. Unknown runtime architecture

- We don't know exactly how clawdbot's sandbox works
 - Could be Node.js child processes, Docker, WebAssembly, or custom isolation
 - Would need to study clawdbot internals
2. **Large binary dependencies**
 - LaTeX distribution: ~200MB
 - pandoc: ~50MB + dependencies
 - Bloats runtime significantly
 3. **Platform-specific binaries**
 - macOS binaries won't work on Linux
 - Need different builds for different platforms
 - ARM vs x86 architectures
 4. **Maintenance burden**
 - Keep binaries updated
 - Handle version conflicts
 - Debug runtime-specific issues
 - Test across platforms
 5. **Security tradeoffs**
 - More binaries = larger attack surface
 - Tools like `pdflatex` can execute arbitrary code
 - Conflicts with sandbox isolation goals
-

1.5 The Plugin Approach

1.5.1 Why Plugins Are Better

Plugins run on the host system, outside the agent sandbox, with full access to system binaries.

1.5.2 Architecture Comparison

1.5.2.1 Without Plugin (Fails)

```
User: "Convert markdown to PDF"
↓
Agent Runtime (Sandbox)
↓
bash("pandoc file.md -o file.pdf")
↓
Error: pandoc not found
```

1.5.2.2 With Plugin (Works!)

```
User: "Convert markdown to PDF"
↓
Agent Runtime (Sandbox)
↓
markdown_to_pdf(input, output) ← Agent calls tool
↓
↓ (breaks out of sandbox via Plugin API)
```

↓
Plugin from ~/.clawdbot/extensions/ (Host System)
↓
exec("pandoc file.md -o file.pdf") ← Runs on host with full PATH
↓
Success! PDF generated
↓
Result returned to agent

1.5.3 How It Works

Host System (Your Mac/Server)

Plugin runs HERE
Has pandoc, pdflatex, git, etc.
Full PATH: /opt/homebrew/bin:/usr/bin:...

Extensions Directory: ~/.clawdbot/extensions/

voice-call/ (installed plugin)
index.ts
package.json
clawdbot.plugin.json

markdown-to-pdf → /path/to/clawdbot/plugins/
(symlink to repo - for development)
index.ts ← Registers markdown_to_pdf tool
package.json
clawdbot.plugin.json
src/
convert.ts ← Conversion logic

Gateway Process

1. Load plugins from extensions/
2. Register tools (markdown_to_pdf)
3. Start agent runtime

Agent Runtime (Sandbox)

User: "Convert report.md to PDF"
↓

```

Agent decides to use markdown_to_pdf tool
↓
Tool call: markdown_to_pdf({
  inputPath: "/workspace/report.md",
  outputPath: "/workspace/report.pdf",
  options: { pdfEngine: "xelatex" }
})
↓

↓ (Plugin API boundary - breaks out of sandbox)

Plugin: markdown-to-pdf (from extensions/)
Location: ~/.clawdbot/extensions/markdown-to-pdf/

1. Receive tool call from agent
2. Validate inputs
3. exec("pandoc input -o output")
   → Runs on HOST system
   → Has access to pandoc binary
   → PATH: /opt/homebrew/bin:/Library/TeX/texbin
4. Return result to agent

```

1.5.4 Benefits of Plugin Approach

Aspect	Installing in Runtime	Plugin Approach
Complexity	High (modify runtime)	Low (just write plugin)
Maintenance	Hard (runtime updates)	Easy (plugin updates)
Platform	Requires builds per platform	Plugin handles platform differences
Binary Size	Bloats runtime	No runtime bloat
Security	Weakens sandbox	Maintains sandbox security
Dependencies	Must bundle everything	Uses system binaries
Updates	Requires runtime rebuild	Update plugin only
Testing	Test entire runtime	Test plugin in isolation

1.6 Creating a Plugin

1.6.1 Plugin Structure

```

your-plugin/
  package.json      # NPM package definition
  clawdbot.plugin.json # Plugin manifest
  index.ts          # Main entry point

```



```

src/
  tool.ts          # Tool implementation
  utils.ts         # Helper functions
  README.md        # Documentation

```

1.6.2 Minimal Plugin Example

1.6.2.1 package.json

```

{
  "name": "@clawdbot/your-plugin",
  "version": "1.0.0",
  "description": "Your plugin description",
  "main": "index.ts",
  "type": "module",
  "keywords": ["clawdbot-plugin"],
  "peerDependencies": {
    "clawdbot": ">=2026.1.0"
  }
}

```

1.6.2.2 clawdbot.plugin.json

```

{
  "id": "your-plugin",
  "uiHints": {
    "someConfig": {
      "label": "Configuration Label",
      "help": "Help text for this config option"
    }
  }
}

```

1.6.2.3 index.ts

```

import type { ClawdbotPluginApi, ClawdbotPluginDefinition } from "clawdbot/plugin-sdk";

export default {
  id: "your-plugin",
  name: "Your Plugin Name",
  description: "What your plugin does",
  version: "1.0.0",

  register: (api: ClawdbotPluginApi) => {
    const { logger } = api;

    logger.info("Registering your-plugin tool");

    api.registerTool({

```

```

name: "your_tool_name",
description: "What your tool does. The agent will see this description.",

parameters: {
  type: "object",
  properties: {
    input: {
      type: "string",
      description: "Input parameter description",
    },
    options: {
      type: "object",
      properties: {
        option1: {
          type: "boolean",
          description: "Optional parameter",
        },
      },
    },
  },
  required: ["input"],
},

execute: async ({ input, options = {} }) => {
  try {
    logger.info(`Executing tool with input: ${input}`);

    // Your tool logic here
    // This code runs on the HOST system!
    const result = await yourFunction(input, options);

    return {
      success: true,
      result: result,
      message: "Tool executed successfully",
    };
  } catch (error) {
    logger.error(`Tool execution failed: ${error}`);
    return {
      success: false,
      error: String(error),
    };
  }
},
});

logger.info("your-plugin tool registered successfully");
},

```

```
} satisfies ClawdbotPluginDefinition;
```

1.6.3 Plugin Installation

1.6.3.1 Local Development

```
# Copy plugin to extensions  
cp -r your-plugin ~/.clawdbot/extensions/  
  
# Enable in config
```

1.6.3.2 Config (~/.clawdbot/clawdbot.json)

```
{  
  "plugins": {  
    "allow": ["your-plugin"],  
    "entries": {  
      "your-plugin": {  
        "enabled": true,  
        "config": {  
          "someConfig": "value"  
        }  
      }  
    },  
    "installs": {  
      "your-plugin": {  
        "source": "local",  
        "spec": "@clawdbot/your-plugin",  
        "installPath": "/Users/you/.clawdbot/extensions/your-plugin",  
        "version": "1.0.0",  
        "installedAt": "2026-02-07T00:00:00.000Z"  
      }  
    }  
  }  
}
```

1.7 Plugin Development Workflow

1.7.1 Development Setup: Using Symlinks

For developing built-in plugins that ship with clawdbot, use **symlinks** instead of copying files. This makes development much faster and keeps the plugin in the clawdbot repository.

1.7.1.1 Why Symlinks? Without Symlinks (Slow):

```
# Edit plugin in repo  
vim /path/to/clawdbot/plugins/my-plugin/index.ts  
  
# Copy to extensions directory
```

```
cp -r plugins/my-plugin ~/.clawdbot/extensions/
```

```
# Restart gateway
```

```
pnpm run gateway:dev
```

```
# Repeat for every change!
```

With Symlinks (Fast):

```
# One-time setup: Create symlink
```

```
ln -s /path/to/clawdbot/plugins/my-plugin ~/.clawdbot/extensions/my-plugin
```

```
# Edit plugin in repo
```

```
vim /path/to/clawdbot/plugins/my-plugin/index.ts
```

```
# Just restart gateway - changes are immediately available!
```

```
pnpm run gateway:dev
```

1.7.1.2 Setting Up Symlink Development 1. Create Plugin in Repo:

```
cd /path/to/clawdbot
```

```
mkdir -p plugins/my-plugin
```

```
cd plugins/my-plugin
```

```
# Create plugin files
```

```
touch index.ts package.json clawdbot.plugin.json README.md
```

2. Create Symlink:

```
# Link from extensions to repo
```

```
ln -s "$(pwd)" ~/.clawdbot/extensions/my-plugin
```

```
# Verify symlink
```

```
ls -la ~/.clawdbot/extensions/
```

```
# Output: my-plugin -> /path/to/clawdbot/plugins/my-plugin
```

3. Configure and Start:

```
// ~/.clawdbot/clawdbot.json
```

```
{  
  "plugins": {  
    "allow": ["my-plugin"],  
    "entries": {  
      "my-plugin": {  
        "enabled": true,  
        "config": {}  
      }  
    }  
  }  
}
```

```
# Start gateway - plugin loads from repo!
pnpm run gateway:dev
```

1.7.1.3 Directory Structure with Symlinks

```
/path/to/clawdbot/          # Clawdbot repository
plugins/
  markdown-to-pdf/          # ← Edit here (version controlled)
    index.ts
    clawdbot.plugin.json
    package.json
    src/convert.ts
  my-new-plugin/            # ← Your plugin here
    index.ts
    ...

~/.clawdbot/extensions/    # Extension directory
markdown-to-pdf → /path/to/clawdbot/plugins/markdown-to-pdf # Symlink!
my-new-plugin → /path/to/clawdbot/plugins/my-new-plugin     # Symlink!
voice-call/                # Installed plugin (not symlinked)
```

1.7.1.4 Benefits

Aspect	Copying Files	Symlinks
Edit & Test	Edit → Copy → Restart	Edit → Restart
Version Control	Must remember to copy	Always in sync
Collaboration	Others need to copy	Just clone & symlink
Ship with Clawdbot	Manual packaging	Already in repo
Mistakes	Can edit wrong copy	Single source of truth
Build Process	Need to script copying	Automatic

1.7.1.5 Development Workflow Example Example: Adding a Feature to markdown-to-pdf Plugin

```
# 1. Edit plugin code (in repo)
vim plugins/markdown-to-pdf/src/convert.ts
# Add support for custom fonts...

# 2. Update tests
vim plugins/markdown-to-pdf/test/convert.test.ts

# 3. Restart gateway (changes automatically available via symlink)
pnpm run gateway:dev

# 4. Test via Telegram/WhatsApp
# "Convert report.md to PDF with custom font"
```

```
# 5. If it works, commit to repo
git add plugins/markdown-to-pdf/
git commit -m "Add custom font support to markdown-to-pdf"
git push
```

6. Done! Plugin is now version controlled and ready to ship

1.7.1.6 When to Use Each Approach Use Symlinks (Built-in Plugins): - Developing plugins that ship with clawdbot - Plugins everyone should have - Core functionality extensions - Examples: markdown-to-pdf, image-processing, code-formatter

Use Installation (User Plugins): - User-specific plugins - Company-internal tools - Third-party plugins from npm - Examples: custom-api-client, internal-database-tool

1.7.1.7 Troubleshooting Symlinks Symlink not working?

```
# Check if symlink exists
ls -la ~/.clawdbot/extensions/my-plugin

# Verify target exists
ls -la /path/to/clawdbot/plugins/my-plugin

# Recreate if broken
rm ~/.clawdbot/extensions/my-plugin
ln -s /path/to/clawdbot/plugins/my-plugin ~/.clawdbot/extensions/my-plugin
```

Gateway not loading plugin?

```
# Check gateway logs
tail -f /tmp/clawdbot/clawdbot-*.log | grep my-plugin

# Run doctor to see plugin status
clawdbot doctor

# Check plugin config
cat ~/.clawdbot/clawdbot.json | jq '.plugins'
```

Changes not appearing?

```
# Make sure you're editing the repo version, not a copy!
which-file-am-i-editing="$(pwd)/index.ts"
symlink-target="$(readlink ~/.clawdbot/extensions/my-plugin)/index.ts"

if [ "$which-file-am-i-editing" = "$symlink-target" ]; then
  echo " Editing correct file"
else
  echo " Editing wrong file!"
  echo "You're editing: $which-file-am-i-editing"
  echo "Should edit: $symlink-target"
fi
```

1.8 Example: Markdown to PDF Plugin

1.8.1 Use Case

Problem: Agent needs to convert markdown files to PDF, but: - `pandoc` not available in agent runtime - `pdflatex` not available in agent runtime - Can't modify runtime to include these binaries

Solution: Create a plugin that runs on the host system.

1.8.2 Plugin Structure

```
markdown-to-pdf/  
  package.json  
  clawdbot.plugin.json  
  index.ts           # Registers markdown_to_pdf tool  
  src/  
    convert.ts       # Conversion logic  
  README.md
```

1.8.3 Key Files

1.8.3.1 index.ts - Tool Registration

```
import type { ClawdbotPluginApi, ClawdbotPluginDefinition } from "clawdbot/plugin-sdk";  
import { convertMarkdownToPdf } from "../src/convert.js";  
  
export default {  
  id: "markdown-to-pdf",  
  name: "Markdown to PDF",  
  description: "Convert markdown files to PDF using Pandoc",  
  version: "1.0.0",  
  
  register: (api: ClawdbotPluginApi) => {  
    api.registerTool({  
      name: "markdown_to_pdf",  
      description: `Convert a markdown file to PDF format.  
  
      This tool requires pandoc and LaTeX on the host system.  
      Supports Unicode characters, code highlighting, and TOC generation.`,  
  
      parameters: {  
        type: "object",  
        properties: {  
          inputPath: {  
            type: "string",  
            description: "Path to input markdown file",  
          },  
          outputPath: {
```

```

        type: "string",
        description: "Path for output PDF file",
    },
    options: {
        type: "object",
        properties: {
            pdfEngine: {
                type: "string",
                enum: ["pdflatex", "xelatex", "lualatex"],
                description: "LaTeX engine to use",
            },
            toc: {
                type: "boolean",
                description: "Generate table of contents",
            },
        },
    },
    required: ["inputPath", "outputPath"],
},

execute: async ({ inputPath, outputPath, options = {} }) => {
    const result = await convertMarkdownToPdf(
        inputPath,
        outputPath,
        options,
        api.logger
    );

    return result;
},
});
} satisfies ClawdbotPluginDefinition;

```

1.8.3.2 src/convert.ts - Conversion Logic

```

import { exec } from "node:child_process";
import { promisify } from "node:util";
import type { PluginLogger } from "clawdbot/plugin-sdk";

const execAsync = promisify(exec);

export async function convertMarkdownToPdf(
    inputPath: string,
    outputPath: string,
    options: any,
    logger: PluginLogger

```



```

) {
  // Ensure LaTeX is in PATH (runs on HOST)
  const PATH = "/opt/homebrew/bin:/Library/TeX/texbin:" + process.env.PATH;

  // Build pandoc command
  const command = `pandoc "${inputPath}" -o "${outputPath}" --pdf-engine=xelatex`;

  logger.info(`Running: ${command}`);

  try {
    // Execute on HOST system - has access to pandoc!
    const { stdout, stderr } = await execAsync(command, {
      env: { ...process.env, PATH },
    });

    return {
      success: true,
      outputPath,
      message: `PDF generated: ${outputPath}`,
    };
  } catch (error) {
    logger.error(`Conversion failed: ${error}`);
    return {
      success: false,
      error: String(error),
    };
  }
}

```

1.8.4 How It Works in Practice

1. User asks agent: “Convert report.md to PDF”
2. Agent decides to use tool:

```

{
  "name": "markdown_to_pdf",
  "arguments": {
    "inputPath": "/Users/john/workspace/report.md",
    "outputPath": "/Users/john/workspace/report.pdf"
  }
}

```

3. Plugin executes on host:

- Runs pandoc command on host system
- Has access to /opt/homebrew/bin/pandoc
- Has access to /Library/TeX/texbin/pdflatex
- Generates PDF successfully

4. Plugin returns result to agent:

```
{
  "success": true,
  "outputPath": "/Users/john/workspace/report.pdf",
  "message": "PDF generated successfully"
}
```

5. Agent tells user: “I’ve converted report.md to PDF. The file is saved at report.pdf”

1.9 Best Practices

1.9.1 When to Create a Plugin

Good use cases: - Need to access system binaries not in agent runtime - Need to interact with system services (Docker, databases) - Need to call external APIs with complex authentication - Need to maintain state across agent sessions - Need to run background services

Don’t need a plugin for: - Simple file operations (use built-in tools) - Basic shell commands (use bash tool) - Pure data transformation (agent can do this) - Simple API calls (agent can do this)

1.9.2 Security Considerations

1. Validate all inputs

```
if (!inputPath || !outputPath) {
  throw new Error("Missing required parameters");
}
```

2. Sanitize file paths

```
import { resolve } from "node:path";
const safePath = resolve(inputPath);
```

3. Limit resource usage

```
const { stdout } = await execAsync(command, {
  timeout: 30000, // 30 second timeout
  maxBuffer: 10 * 1024 * 1024, // 10MB buffer
});
```

4. Log security-relevant events

```
logger.warn(`Executing system command: ${command}`);
```

1.9.3 Error Handling

```
execute: async (params) => {
  try {
    // Validate inputs
    if (!params.inputPath) {
      return {
```

```

        success: false,
        error: "Missing input path",
    };
}

// Check file exists
try {
    await access(params.inputPath, constants.R_OK);
} catch {
    return {
        success: false,
        error: `Input file not found: ${params.inputPath}`,
    };
}

// Perform operation
const result = await yourOperation(params);

// Return success
return {
    success: true,
    result: result,
    message: "Operation completed successfully",
};

} catch (error) {
    // Log error
    logger.error(`Tool failed: ${error}`);

    // Return user-friendly error
    return {
        success: false,
        error: error instanceof Error ? error.message : String(error),
    };
}
}

```

1.10 Summary

1.10.1 Key Takeaways

1. Agent runtime is **sandboxed** for security
2. System binaries aren't available in the sandbox
3. Installing in runtime is **complex** and has drawbacks
4. Plugins run on host with full system access
5. Plugins are the **right approach** for system integration

1.10.2 Architecture Benefits

Requirement	Runtime Installation	Plugin Approach
Access system binaries	Complex	Simple
Maintain security	Weakened	Preserved
Platform independence	Hard	Easy
Easy maintenance	Hard	Easy
Small footprint	Large	Small
Quick to develop	Slow	Fast

1.10.3 When to Use Each Approach

Use plugins when: - Need to call system binaries (pandoc, git, docker) - Need to interact with system services - Need to maintain security isolation - Want easy maintenance and updates

Modify runtime when: - Only if you control the entire deployment - Only if security isn't a concern - Only if you need the binary in ALL agent contexts - Generally: **don't do this**

1.11 Resources

- Clawdbot Plugin SDK Documentation
 - Plugin Examples
 - Security Best Practices
-

1.12 Questions?

If you have questions about plugin development:

1. Check existing plugins in `~/.clawdbot/extensions/`
2. Read the Plugin SDK types in `src/plugin-sdk/index.ts`
3. Open an issue on GitHub
4. Ask in the Clawdbot community

Happy plugin development!